

ATIVIDADE AVALIATIVA - ESTRUTURAS DE DADOS

Arrays, Matrizes, Algoritmos de Ordenação e Busca

Aluno: Igor Carvalho de Oliveira

Disciplina: Estruturas de Dados

Curso: Ciência da Computação

Valor: 1,0 ponto

Atividade desenvolvida em Python, contemplando pesquisa teórica, implementação, experimentos, contagem de operações, análise de complexidade e conclusão.

PARTE 1 - PESQUISA: BUBBLE SORT E QUICK SORT

1. Bubble Sort

O Bubble Sort é um algoritmo de ordenação baseado em comparações entre elementos vizinhos. O algoritmo percorre o array comparando dois elementos consecutivos. Quando o elemento da esquerda é maior que o elemento da direita, os dois são trocados. O processo é repetido até que os elementos estejam na ordem correta.

Melhor caso: $O(n)$, quando o array já está ordenado e a implementação utiliza uma verificação para interromper o algoritmo quando nenhuma troca ocorre.

Caso médio: $O(n^2)$.

Pior caso: $O(n^2)$.

Memória: $O(1)$, pois a ordenação é realizada no próprio array.

Vantagens: implementação simples, fácil compreensão, pouca memória adicional e utilidade didática.

Limitações: grande quantidade de comparações e trocas para conjuntos grandes de dados.

Uso adequado: arrays pequenos, atividades didáticas e situações em que a simplicidade é mais importante que o desempenho.

Uso não recomendado: grandes conjuntos de dados ou aplicações que necessitam de alto desempenho.

2. Quick Sort

O Quick Sort utiliza a estratégia de divisão e conquista. Um elemento é escolhido como pivô e o array é reorganizado de modo que os elementos menores ou iguais ao pivô fiquem de um lado e os maiores do outro. Em seguida, o procedimento é aplicado recursivamente às duas partes.

Melhor caso: $O(n \log n)$, quando o pivô divide o conjunto em partes aproximadamente equilibradas.

Caso médio: $O(n \log n)$.

Pior caso: $O(n^2)$, quando as divisões são muito desequilibradas.

Memória: $O(\log n)$ em média devido à recursão, podendo chegar a $O(n)$ no pior caso.

Vantagens: bom desempenho médio, adequado para grandes conjuntos e baixo uso de memória adicional em implementações in-place.

Limitações: depende da escolha do pivô e pode atingir $O(n^2)$ no pior caso.

Uso adequado: grandes conjuntos de dados e situações que exigem bom desempenho médio.

Uso não recomendado: quando a estabilidade é obrigatória ou quando a escolha do pivô pode produzir partições muito desequilibradas sem tratamento.

3. Tabela comparativa

Característica	Bubble Sort	Quick Sort
Princípio de funcionamento	Compara elementos vizinhos e realiza trocas.	Divide o array utilizando um pivô e ordena as partes recursivamente.
Melhor caso	$O(n)$ com otimização.	$O(n \log n)$.
Caso médio	$O(n^2)$.	$O(n \log n)$.
Pior caso	$O(n^2)$.	$O(n^2)$.
Uso de memória	$O(1)$.	$O(\log n)$ médio / $O(n)$ pior caso.
Vantagem principal	Simplicidade.	Alto desempenho médio.
Limitação principal	Ineficiente em arrays grandes.	Pode atingir $O(n^2)$ dependendo do pivô.
Aplicação recomendada	Arrays pequenos e situações didáticas.	Arrays grandes e aplicações que exigem desempenho.

PARTE 2 - EXPERIMENTO DE ORDENAÇÃO

1. Metodologia

Foram utilizados arrays de 10, 20 e 1.000 elementos. Para garantir uma comparação justa, os dois algoritmos recebem exatamente os mesmos dados. A geração é determinística pela fórmula $\text{valor}[i] = (37 \times i + 17) \bmod n$. Depois da geração, o array original é copiado para as versões utilizadas pelo Bubble Sort e pelo Quick Sort.

No Bubble Sort foram contabilizadas comparações e trocas. No Quick Sort foram contabilizadas comparações e movimentações. Para esta atividade, cada troca de dois elementos no Quick Sort é considerada como três movimentações, correspondentes às três atribuições de uma troca com variável temporária.

2. Código Python - Bubble Sort e Quick Sort

```
class ResultadoBubble:
    def __init__(self):
        self.comparacoes = 0
        self.trocas = 0

class ResultadoQuick:
    def __init__(self):
        self.comparacoes = 0
        self.movimentacoes = 0

def gerar_array(n):
    return [(37 * i + 17) % n for i in range(n)]

def bubble_sort(v):
    resultado = ResultadoBubble()
    n = len(v)

    for i in range(n - 1):
        houve_troca = False

        for j in range(n - 1 - i):
            resultado.comparacoes += 1

            if v[j] > v[j + 1]:
                v[j], v[j + 1] = v[j + 1], v[j]
                resultado.trocas += 1
                houve_troca = True

        if not houve_troca:
            break

    return resultado

def trocar(v, i, j, resultado):
    if i != j:
        v[i], v[j] = v[j], v[i]
        resultado.movimentacoes += 3

def particionar(v, inicio, fim, resultado):
    pivo = v[fim]
    i = inicio

    for j in range(inicio, fim):
        resultado.comparacoes += 1

        if v[j] <= pivo:
            trocar(v, i, j, resultado)
            i += 1

    trocar(v, i, fim, resultado)
    return i

def quick_sort_recursivo(v, inicio, fim, resultado):
    if inicio < fim:
        p = particionar(v, inicio, fim, resultado)
        quick_sort_recursivo(v, inicio, p - 1, resultado)
        quick_sort_recursivo(v, p + 1, fim, resultado)

def quick_sort(v):
    resultado = ResultadoQuick()
    quick_sort_recursivo(v, 0, len(v) - 1, resultado)
    return resultado

tamanhos = [10, 20, 1000]

print("TAMANHO | BUBBLE COMP. | BUBBLE TROCAS | QUICK COMP. | QUICK MOV.")

for tamanho in tamanhos:
    original = gerar_array(tamanho)

    array_bubble = original.copy()
    array_quick = original.copy()

    resultado_bubble = bubble_sort(array_bubble)
    resultado_quick = quick_sort(array_quick)

    print(
        f"{tamanho:7} | "
        f"{resultado_bubble.comparacoes:12} | "
        f"{resultado_bubble.trocas:14} | "
        f"{resultado_quick.comparacoes:11} | "
        f"{resultado_quick.movimentacoes:11}"
    )
```

3. Resultados do experimento

Tamanho	Bubble - Comparações	Bubble - Trocas	Quick - Comparações	Quick - Movimentações
10	45	27	26	33
20	190	127	88	87
1.000	498.554	243.027	19.167	43.071

4. Análise dos resultados

- a) Para 10 elementos, o Quick Sort realizou menos comparações: 26 contra 45 do Bubble Sort.
- b) Para 20 elementos, o comportamento permaneceu favorável ao Quick Sort: 88 comparações contra 190 do Bubble Sort.
- c) Para 1.000 elementos, a diferença aumentou muito. O Bubble Sort realizou 498.554 comparações e 243.027 trocas, enquanto o Quick Sort realizou 19.167 comparações e 43.071 movimentações.
- d) O Bubble Sort apresentou o maior crescimento da quantidade de operações.
- e) Sim. Os resultados são coerentes com as complexidades teóricas: Bubble Sort possui comportamento médio $O(n^2)$, enquanto Quick Sort possui comportamento médio $O(n \log n)$.
- f) Eu escolheria Bubble Sort para arrays pequenos, situações didáticas ou quando a simplicidade for prioridade.
- g) Eu escolheria Quick Sort para conjuntos maiores de dados quando for necessário obter bom desempenho médio.

PARTE 3 - INVESTIGAÇÃO DE BUSCA EM MATRIZES

1. Busca sequencial

A busca sequencial verifica cada posição da matriz até encontrar o valor procurado. Como a matriz possui duas dimensões, são utilizados dois loops aninhados: o primeiro percorre as linhas e o segundo percorre as colunas. A posição é representada por `matriz[i][j]`, em que `i` representa a linha e `j` representa a coluna.

2. Código Python

```
def criar_matriz(linhas, colunas):
    matriz = []
    valor = 1

    for i in range(linhas):
        linha = []

        for j in range(colunas):
            linha.append(valor)
            valor += 1

        matriz.append(linha)

    return matriz

def busca_sequencial(matriz, valor):
    comparacoes = 0

    for i in range(len(matriz)):
        for j in range(len(matriz[i])):
            comparacoes += 1

            if matriz[i][j] == valor:
                return True, i, j, comparacoes

    return False, -1, -1, comparacoes

def executar_teste(linhas, colunas):
    matriz = criar_matriz(linhas, colunas)
    quantidade_elementos = linhas * colunas

    inicio = busca_sequencial(matriz, 1)
    final = busca_sequencial(matriz, quantidade_elementos)
    inexistente = busca_sequencial(matriz, -1)

    print(f"\nMatriz {linhas} x {colunas}")
    print(f"Busca no início: {inicio[3]} comparação(ões)")
    print(f"Busca no final: {final[3]} comparação(ões)")
    print(f"Valor inexistente: {inexistente[3]} comparação(ões)")

executar_teste(2, 2)
executar_teste(10, 10)
executar_teste(100, 100)
```

3. Resultados

Matriz	Elementos	Início	Final	Inexistente
2 × 2	4	1	4	4
10 × 10	100	1	100	100
100 × 100	10.000	1	10.000	10.000

4. Análise

- Encontrar um elemento no início exige menos operações porque a busca começa pela primeira posição. Assim, apenas uma comparação é necessária.
- Quando o elemento não existe, todas as posições precisam ser verificadas para confirmar sua ausência.
- O pior caso ocorre quando o elemento está na última posição ou não está presente. Nesses casos, a matriz inteira pode ser percorrida.
- O aumento das dimensões aumenta a quantidade de posições. Uma matriz 2 × 2 possui 4 elementos, uma 10 × 10 possui 100 e uma 100 × 100 possui 10.000.
- Para uma matriz com m linhas e n colunas, a complexidade da busca sequencial é $O(m \times n)$.

PARTE 4 - HANDS ON 1: INVESTIGAÇÃO DO ARRAY

1. Código Python

```
temperaturas = []

for i in range(10):
    temperatura = float(input(f"Digite a temperatura {i}: "))
    temperaturas.append(temperatura)

print("\nTEMPERATURAS ARMAZENADAS")

for i in range(10):
    print(f"Índice {i}: {temperaturas[i]:.2f} °C")

soma = 0

for temperatura in temperaturas:
    soma += temperatura

media = soma / 10

maior = temperaturas[0]
menor = temperaturas[0]
indice_maior = 0
indice_menor = 0

for i in range(1, 10):
    if temperaturas[i] > maior:
        maior = temperaturas[i]
        indice_maior = i

    if temperaturas[i] < menor:
        menor = temperaturas[i]
        indice_menor = i

acima_media = 0

for temperatura in temperaturas:
    if temperatura > media:
        acima_media += 1

print("\nRESULTADOS")
print(f"Média: {media:.2f} °C")
print(f"Maior temperatura: {maior:.2f} °C")
print(f"Índice do maior: {indice_maior}")
print(f"Menor temperatura: {menor:.2f} °C")
print(f"Índice do menor: {indice_menor}")
print(f"Valores acima da média: {acima_media}")
```

2. Exemplo de execução

Índice	Temperatura
0	19,5 °C
1	21,0 °C
2	18,2 °C
3	25,3 °C
4	23,6 °C
5	27,1 °C
6	22,4 °C
7	24,8 °C
8	20,9 °C
9	26,0 °C

Resultados: média = 22,88 °C; maior = 27,10 °C; índice do maior = 5; menor = 18,20 °C; índice do menor = 2; valores acima da média = 5.

Considerando os percursos do programa, são aproximadamente 30 posições percorridas: 10 para entrada e análise, 10 para exibição e 10 para contar os valores acima da média. A complexidade é $O(n)$, pois o crescimento é linear.

PARTE 5 - HANDS ON 2: MATRIZ APLICADA - MONITORAMENTO DE SENSORES

1. Código Python

```
SENSORES = 5
HORAS = 24

sensores = []

for i in range(SENSORES):
    linha = []

    for j in range(HORAS):
        temperatura = float(
            input(f"Sensor {i} - Hora {j}: ")
        )
        linha.append(temperatura)

    sensores.append(linha)

medias = []

for i in range(SENSORES):
    soma = 0

    for j in range(HORAS):
        soma += sensores[i][j]

    media = soma / HORAS
    medias.append(media)

maior = sensores[0][0]
sensor_maior = 0
horario_maior = 0
soma_geral = 0

for i in range(SENSORES):
    for j in range(HORAS):
        temperatura = sensores[i][j]
        soma_geral += temperatura

        if temperatura > maior:
            maior = temperatura
            sensor_maior = i
            horario_maior = j

total_medicoes = SENSORES * HORAS
media_geral = soma_geral / total_medicoes

limite = float(
    input("\nDigite o limite de temperatura: ")
)

acima_limite = 0

for i in range(SENSORES):
    for j in range(HORAS):
        if sensores[i][j] > limite:
            acima_limite += 1

print("\n===== RESULTADOS =====")
print("\nMÉDIA DE CADA SENSOR")

for i in range(SENSORES):
    print(f"Sensor {i}: {medias[i]:.2f} °C")

print(f"\nMaior temperatura: {maior:.2f} °C")
print(f"Sensor responsável: {sensor_maior}")
print(f"Horário: {horario_maior}:00")
print(f"Média geral: {media_geral:.2f} °C")
print(f"Leituras acima do limite: {acima_limite}")
```

2. Explicação

Os loops aninhados são necessários porque a matriz possui duas dimensões. O primeiro loop percorre as linhas, que representam os sensores, e o segundo percorre as colunas, que representam as 24 horas.

O acesso `sensores[i][j]` identifica uma medição específica. O índice `i` representa o sensor e `j` representa o horário. Como existem 5 sensores e 24 horários, são $5 \times 24 = 120$ posições.

Assim, o percurso completo realiza 120 visitas às posições. De forma geral, para uma matriz com `m` linhas e `n` colunas, a complexidade é $O(m \times n)$.

3. Exemplo de dados e resultados

Para apresentar um exemplo reproduzível sem exigir a digitação de 120 valores, pode-se considerar a regra: $\text{temperatura} = 20 + (\text{sensor} \times 1,5) + (\text{hora} \times 0,35)$.

Sensor	Média
0	24,025 °C

Sensor	Média
1	25,525 °C
2	27,025 °C
3	28,525 °C
4	30,025 °C

Resultado geral do exemplo: maior temperatura = 34,05 °C; sensor responsável = 4; horário = 23 horas; média geral = 27,025 °C. Utilizando limite de 28 °C, há 46 leituras acima do limite.

PARTE 6 - ANÁLISE E CONCLUSÃO

1. O aumento do tamanho da estrutura influencia a quantidade de operações?

Sim. Os experimentos demonstraram que o aumento da quantidade de dados influencia diretamente o número de operações. Na busca sequencial, uma matriz 2×2 possui 4 posições, enquanto uma matriz 100×100 possui 10.000. No pior caso, todas essas posições podem precisar ser verificadas.

2. Bubble Sort e Quick Sort crescem da mesma maneira?

Não. O Bubble Sort apresenta complexidade média $O(n^2)$, enquanto o Quick Sort apresenta complexidade média $O(n \log n)$. Por isso, o aumento do tamanho da entrada produz um crescimento muito maior no número de operações do Bubble Sort.

3. Por que somente o resultado final não é suficiente?

Porque dois algoritmos podem produzir exatamente o mesmo resultado e ainda assim utilizar quantidades muito diferentes de operações. No experimento, os dois algoritmos ordenaram corretamente os arrays, mas o Bubble Sort realizou uma quantidade muito maior de comparações e trocas no array de 1.000 elementos.

Conclusão geral

Os experimentos demonstraram na prática a relação entre tamanho da entrada, quantidade de operações e complexidade computacional. O Bubble Sort mostrou-se adequado para conjuntos pequenos e para fins didáticos, mas seu crescimento quadrático tornou-se evidente com 1.000 elementos. O Quick Sort apresentou desempenho significativamente melhor no experimento, coerente com sua complexidade média $O(n \log n)$.

A busca sequencial em matrizes também demonstrou que o aumento das dimensões influencia diretamente a quantidade de comparações. Para uma matriz 100×100 existem 10.000 posições e, no pior caso, todas precisam ser verificadas.

O Hands On 1 demonstrou o uso de arrays para armazenar e analisar temperaturas. O Hands On 2 mostrou como uma matriz bidimensional pode representar medições de sensores ao longo das horas do dia.

Portanto, a análise de algoritmos não deve considerar somente se o resultado está correto. Também é necessário observar quantas operações são realizadas para obtê-lo. A relação principal observada nesta atividade é: Tamanho da entrada $\rightarrow$ Número de operações $\rightarrow$ Complexidade $\rightarrow$ Eficiência.

RESULTADOS QUE DEVERÃO SER APRESENTADOS

Item exigido	Atendimento
Pesquisa sobre Bubble Sort	Incluído na Parte 1.
Pesquisa sobre Quick Sort	Incluído na Parte 1.
Comparação entre algoritmos	Tabela comparativa incluída.
Código Bubble Sort	Código Python incluído.
Código Quick Sort	Código Python incluído.
Arrays 10, 20 e 1.000	Testes e tabela incluídos.
Tabela de operações	Incluída na Parte 2.
Código da busca em matrizes	Incluído na Parte 3.
Matrizes 2×2 , 10×10 e 100×100	Testes e tabela incluídos.
Tabela de operações da busca	Incluída na Parte 3.
Hands On 1	Array de 10 temperaturas incluído.
Hands On 2	Matriz de 5 sensores $\times$ 24 horas incluída.
Análise dos resultados	Incluída nas Partes 2, 3 e 6.
Conclusão	Incluída na Parte 6.

REFERÊNCIAS

CORMEN, Thomas H.; LEISERSON, Charles E.; RIVEST, Ronald L.; STEIN, Clifford. Introduction to Algorithms. 4. ed. Cambridge: MIT Press, 2022.

SEEDGEWICK, Robert; WAYNE, Kevin. Algorithms. 4. ed. Boston: Addison-Wesley, 2011.

ZIVIANI, Nivio. Projeto de Algoritmos: com Implementações em Java e C++. 3. ed. São Paulo: Cengage Learning, 2010.